CS402 – High-Performance Computing

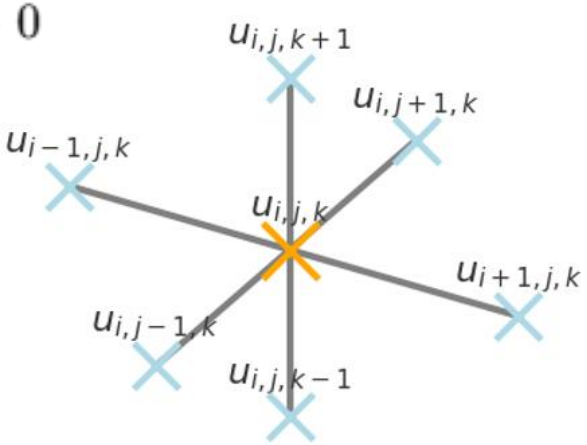
Fundamental Ideas – Parallelism and Systems

Gihan Mudalige
Professor
Department of Computer Science
University of Warwick
g.mudalige@warwick.ac.uk

TIME-TO-SOLUTION

$$\frac{\partial u}{\partial t} = \alpha\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}\right) \quad \text{in } (x,y,z) \in \Omega,\ t > 0$$

$$\begin{aligned} u^{n+1}_{i,j,k} = u^n_{i,j,k} &+ r_x\left(u^n_{i+1,j,k} - 2u^n_{i,j,k} + u^n_{i-1,j,k}\right) \\ &+ r_y\left(u^n_{i,j+1,k} - 2u^n_{i,j,k} + u^n_{i,j-1,k}\right) \\ &+ r_z\left(u^n_{i,j,k+1} - 2u^n_{i,j,k} + u^n_{i,j,k-1}\right), \end{aligned}$$



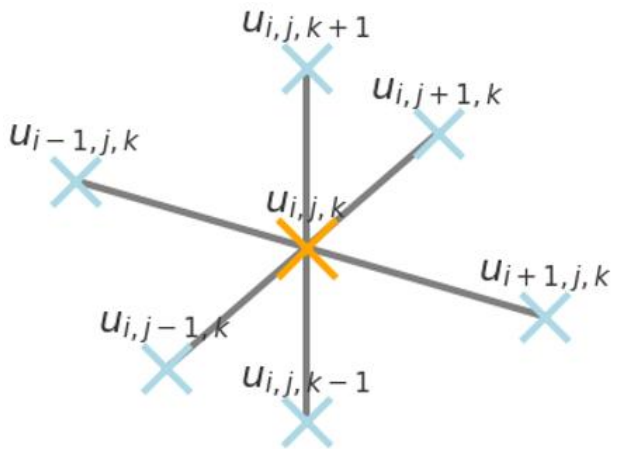
- Time to solution
  - FLOP cost per per grid point : 9 adds and 6 mults = 15 FLOPs
  - Per time step FLOPs: $8 \times 10^9 \times 15 \approx 120$ GFLOPs
  - 4 GHz CPU (≈1 flop/cycle): 4 GFLOP/s → 120/4 ≈ 30 sec/step

  - **SIMD/Vector units** (SSE, AVX, AVX-512) : apply one instruction to 4-16 doubles at once
  - **FMA instructions**: do a multiply and an add in a single operation

  - With AVX2 on doubles: 256-bit registers hold 4 doubles, 1x FMA instruction = 8FLOPs (4 mults + 4 adds)
  - CPU cores can issue 2 FMAs per cycle → 16 FLOPs/cycle. (e.g. AMD Zen 2/3/4, Intel Sapphire Rapids)
  - But assuming an 8 FLOPs/cycle conservative estimate one core will give : 4GHz × 8 FLOP/cycle = 32 GFLOP/s → 120/32 ≈ <u>4 sec/step</u>

TIME-TO-SOLUTION

$$
\begin{aligned}
u^{n+1}_{i,j,k} = u^n_{i,j,k} &+ r_x\left(u^n_{i+1,j,k} - 2u^n_{i,j,k} + u^n_{i-1,j,k}\right) \\
&+ r_y\left(u^n_{i,j+1,k} - 2u^n_{i,j,k} + u^n_{i,j-1,k}\right) \\
&+ r_z\left(u^n_{i,j,k+1} - 2u^n_{i,j,k} + u^n_{i,j,k-1}\right),
\end{aligned}
$$



- But time per time step is bandwidth bound
  - Each update touches (roughly) 7 reads + 1 write
    = 8 doubles ≈ 64 bytes / update (worst case)
  - With good caching (neighbours get reused) : 1 reads + 1 write
    = 2 doubles ≈ 16 bytes / update (best case)
  - Per time step traffic = 2 to 8 × 10^9 × 64 bytes ≈ 128 GB to 512 GB

  - However on a single CPU sustainable single-thread memory bandwidth is often 20GB/s to 50GB/s (per socket BW is higher, but single core typically will not saturate it)
  - At 20GB/s → 128/20 to 512/20 ≈ <u>6.4 sec/step to 25.6 sec/step</u>
  - At 50GB/s → 128/50 to 512/50 ≈ <u>2.56 sec/step to 10.2 sec/step</u>
  - Therefore <u>time to solution</u> for 1000 time steps = **0.71 to 7.1 hours !**

Time-to-solution is intractable/unrealistic even for this very small problem !

Time-to-Solution and Scientific Expressiveness

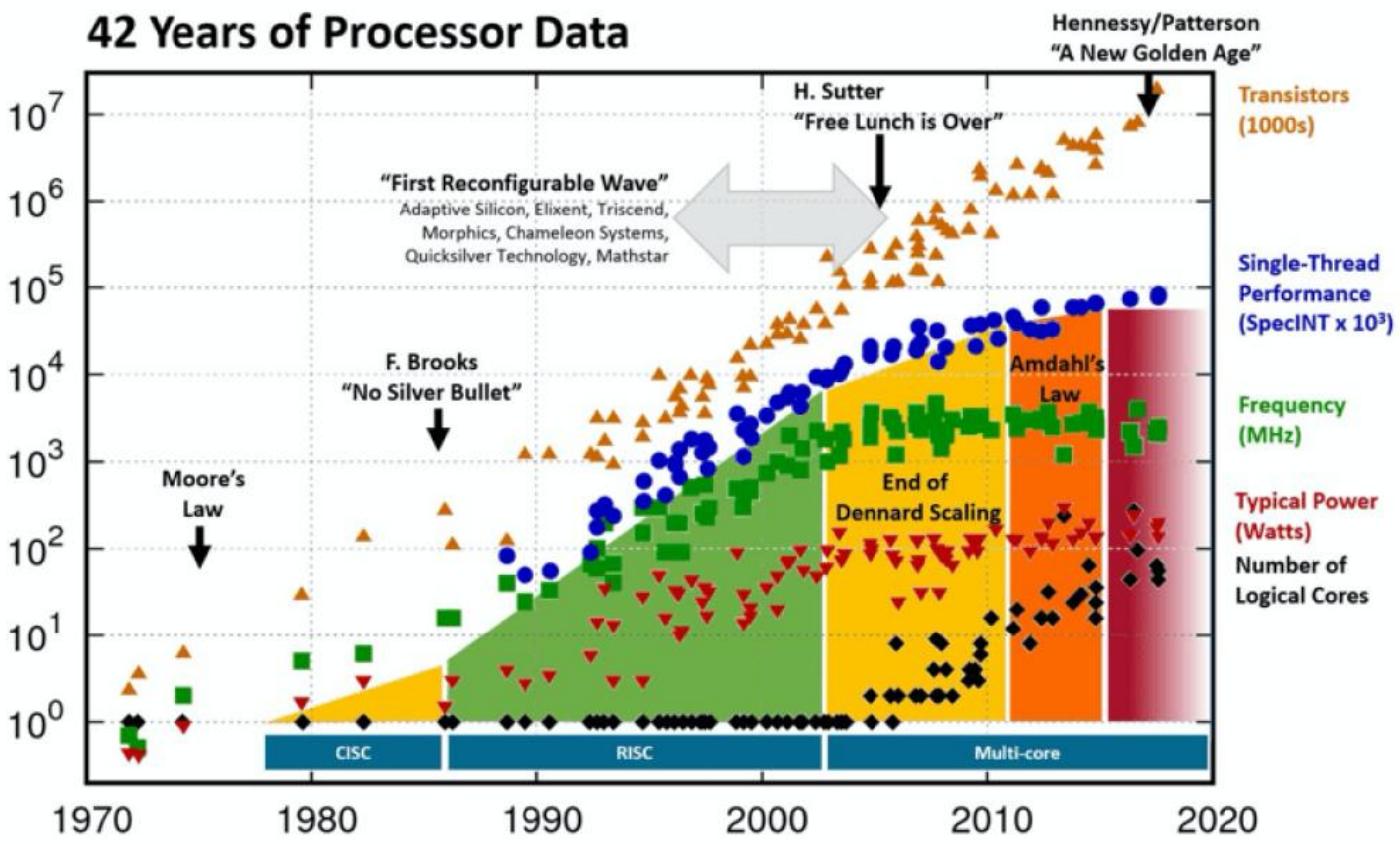
The expressiveness of simulations depends on the availability of compute power

- A computer is **"more useful"** the faster it delivers results – enabling scientists to :
  - Obtain results sooner (shorter time-to-solution)
  - Run more simulations per unit time
  - Explore a larger design or parameter space

- However, in practice, faster compute often lead to :
  - More complex or higher-resolution experiments
  - Increased model fidelity and data volume

- Performance either accelerates discovery (**faster workflows**) or expands the **expressiveness of simulations**
  - **Expressiveness** - *the ability of a simulation/computational model to express more aspects of a real system.*

Moore's Law

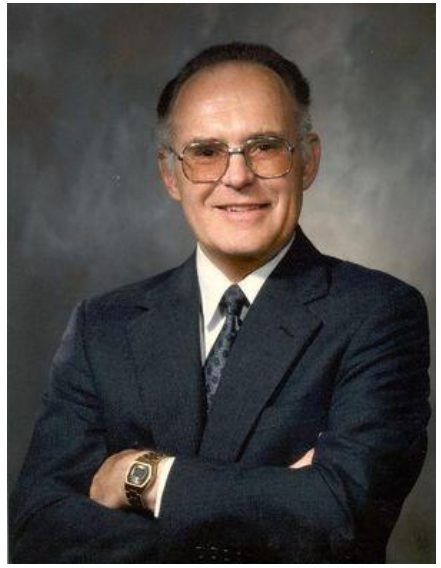


Hennessy and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"
https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/;"First Wave" added by Les Wilson, Frank Schirrmeister
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

- For decades performance improved **automatically**
  - Each new chip generation ran **existing code faster** without modification : (incorrectly) called **Moore's Law**
- But **Moore's Law** is actually about - Transistor density and economics. Not directly about speed.

Moore's Law

- Gordon Moore (Co-founder of Intel) in 1965 observed (revised 1975)
  - Cost per transistor decreases as **density** (transistors per silicon unit area) rises – up to an optimal point
  - At this "sweet spot", the most economical chips double in transistor count approx. **18-24 months**
  - This **exponential** growth held for **~40 years**
  - Primarily due to **economic law of scaling**, enabled by Physics



- It does not say more transistors ≠ more speed, however :
  - higher density → smaller transistors → faster clocks : due to **Dennard's Scaling**
  - Historically this led to : faster clocks → faster processors !

- Clock frequency increase happened from ~**1970 to 2005**

Processor speed increase in this manner no longer happens - due to reasons we will see soon.
However, increase of transistor density still continues (but with diminishing economic returns) !

Dennard (and colleagues) Scaling (1974)

- The power cost P to drive a transistor follows approximately:

$$P = \alpha\, C\, f\, V^2$$

Where, C – *capacitance*, F – *frequency*, V – *voltage*.
α – activity factor (fraction of transistors switching per cycle)

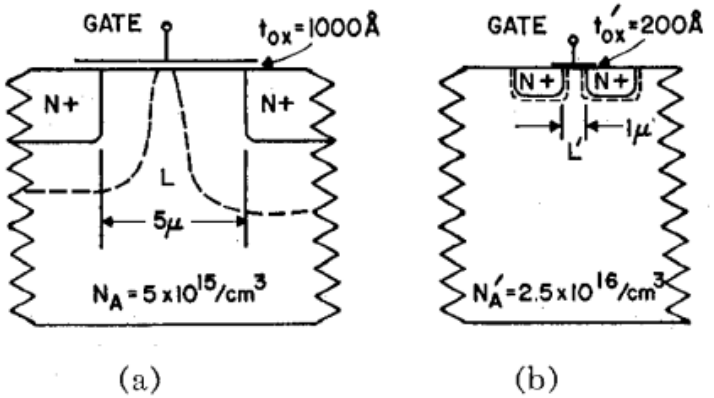


Fig. 1. Illustration of device scaling principles with $\kappa = 5$. (a) Conventional commercially available device structure. (b) Scaled-down device structure.

- When transistor dimension shrink by a factor k
  - Capacitance (C) ↓ (since the gates are smaller)
  - Voltage (V) ↓ (operate at lower voltage because the gates are shorter)
  - Power per transistor (P) ↓ roughly by $1/k^2$
  - **But** you can now fit k^2 times more transistors per chip (since area also shrinks by $1/k^2$)

- In other words, smaller the transistor size → lower C and V : Then for a fixed P we can increase f
- This gave rise to an era (1970 – 2005) of **frequency scaling** – processors going from **MHz to GHz**
  - Double transistor count (Moore's Law), run them at higher f (Dennard scaling) but keep total chip power roughly constant !
  - CPUs in **1980s** – ~10MHz, **1990** - ~100s of MHz, **Early 2000s** – over 3GHz

Power Wall - But they were, all of them, deceived, for another term existed !

$$P = \alpha\, C\, f\, V^2 + I_{leak} V$$

- Dennard's ideal assumption – power density stays constant as we shrink transistors. Relied on:

$$V \propto feature\ size \text{ and } I_{leak} \approx 0$$

- However, as transistor dimensions shrink,
  - Voltage (*V*) cannot scale down indefinitely - limited by noise margins and variability
  - But I_{leak} **rises exponentially** with smaller gate lengths and thinner oxides

- Thus, even when transistors are *idle*, **they dissipate power**
  - Static (leakage) power now accounts for a **large fraction of chip power** – 30-40% of modern chips

Simply pushing frequency f higher now causes overheating !

Power Wall – But its worse in practice !

$$P = \alpha\, C\, f\, V^2 + I_{leak} V$$

- In real chips when you increase **frequency**, you usually also need to **raise the supply voltage (V)** to keep the transistors switching reliably and on time - faster transitions need higher drive current.

f	V	*Relative power* ($P \propto f\, V^2$)
2GHz	1.0 V	1.0x
3GHz	1.1 V	$(1.1^2 \times 3/2) \approx 1.8\times$
4GHz	1.2 V	$(1.2^2 \times 4/2) \approx 2.9\times$

- So, pushing from 2GHz → 4GHz nearly **triples** power — while performance only doubles : The beginning of the **Power Wall**.
- For both chip designers and computing centres buying and running chips, controlling power envelope of a chip is absolutely essential :
  - Purchasing power/energy is costly
  - Chip **"converts" power into heat** – need to get heat out which in turn require more **power for cooling**

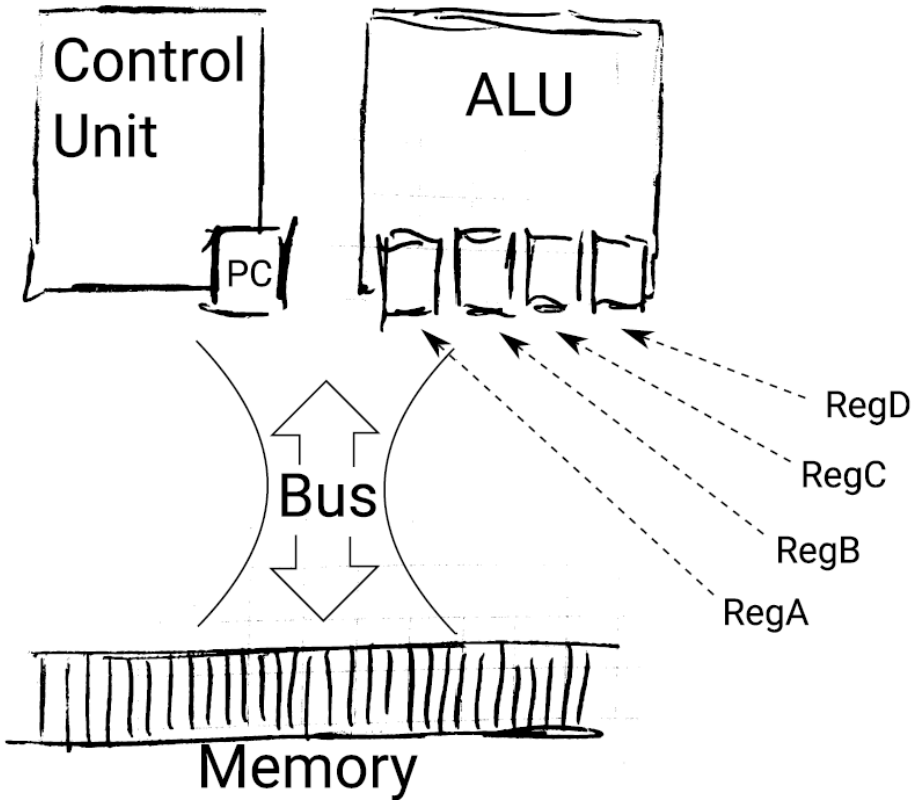
Parallelism replaced frequency scaling as the route to performance.

Machine Model of Execution – von Neumann Architecture

- All modern computers (even parallel ones) derive from the **von Neumann architecture**
  - **Control unit –** orchestrates the workflow
  - **ALU (Arithmetic Logic Unit)** – executes arithmetic & logical operations
  - **Main Memory** – stores both program *and* data
  - **Bus** – interconnects components and peripherals

- The **ALU has registers** — small, fast local storage for temporary values.

- Execution proceeds **in discrete steps (clock cycles)**



Machine Model of Execution – von Neumann Architecture

- High-level code : translate to machine code
- Example : Compute Scalar Product

Execution is **SISD** (Single Instruction, Single Data)
— one operation at a time

$$f = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

```
double f = 0.0;
double x[...]; double y[...];
for (int i=0; i<I; i++) {
  f += x[i] * y[i];
}
```

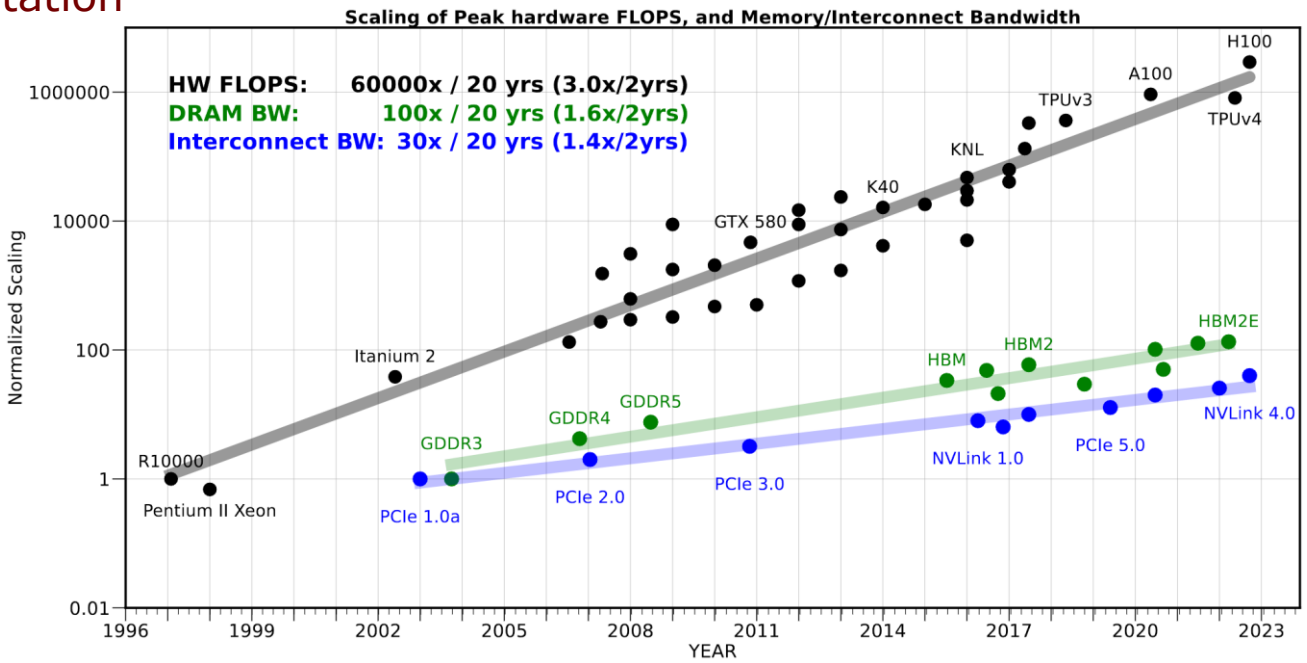
```
00: load 50
 into PC
01: x[0]
02: x[1]
03: x[2]
04: x[3]
05: x[4]
06: x[5]
07: x[6]
08: x[7]
09: x[8]
10: x[9]
11: y[0]
12: y[1]
13: y[2]
14: y[3]
15: y[4]
16: y[5]
17: y[6]
18: y[8]
19: y[9]
20: f
21: i

[...]

50: load 0  into RegD
51: store RegD into
  mem[20]
52: store RegD into
  mem[21]
53: load 1  into RegA
54: load 11 into RegB
55: load mem[RegA]
  into RegC
56: load mem[RegB]
  into RegD
57: mult RegC and
  RegD into RegC
58: load mem[20] into RegD
59: add RegC and RegD
  into RegD
60: store RegD into mem[20]
61: load 1  into RegC
62: load mem[21] into RegD
63: add RegA and RegC
  into RegA
64: add RegB and RegC
  into RegB
65: add RegD and RegC
  into RegD
66: store RegD into mem[21]
67: load 10 into RegC
68: compare RegD and RegC
  into Reg D
69: if equal skip next line
70: load 54 into PC
```

VON NEUMANN ARCHITECTURE: SEQUENTIAL WITH BOTTLENECKS

- Memory bound codes
  - ALUs are fast, memory is slow → CPU *waits* for data
  - Performance limited by data transfer, not computation

- Compute-bound codes : less common

- von Neumann Bottleneck:
  - Data and instructions share one bus → latency and bandwidth limits
  - Data movement dominates power consumption



https://github.com/amirgholami/ai_and_memory_wall/blob/main/imgs/pdfs/hw_scaling.pdf

- Sequential Execution:
  - SISD - Only one instruction acts on one data item at a time
  - No inherent parallelism

To overcome these limits, we must add parallelism – first within the ALU (vectorization), then across cores and nodes

Flynn's Taxonomy (1966) – The Classical view of (Parallel) Systems

Data Streams \ Instruction Streams	Single	Multiple
Single	SISD - Single Instruction Single Data Traditional von Neumann Architecture : Single threaded CPU	MISD – Multiple Instructions Single Data Pipelined systems : FPGAs (?)
Multiple	SIMD - Single Instruction Multiple Data Vector processors	MIMD – Multiple Instructions Multiple Data Multi computers: Clusters

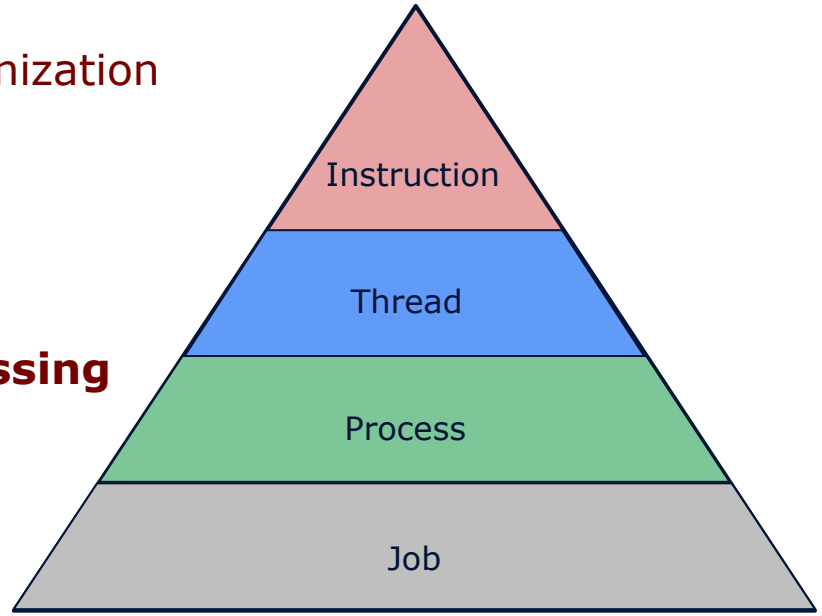
Three Levels of Parallelism : From ALU to Cluster

With Moore's law continuing to hold and a break-down of Dennard scaling the only way to increase performance of computers (with current CMOS technology) is by adding parallelism

1. Vector / SIMD
   - Same instruction applied to **many data elements simultaneously** (data-parallel)
   - Vector units inside cores, GPU warps
   - Programming Model : Compiler auto-vectorization, intrinsics

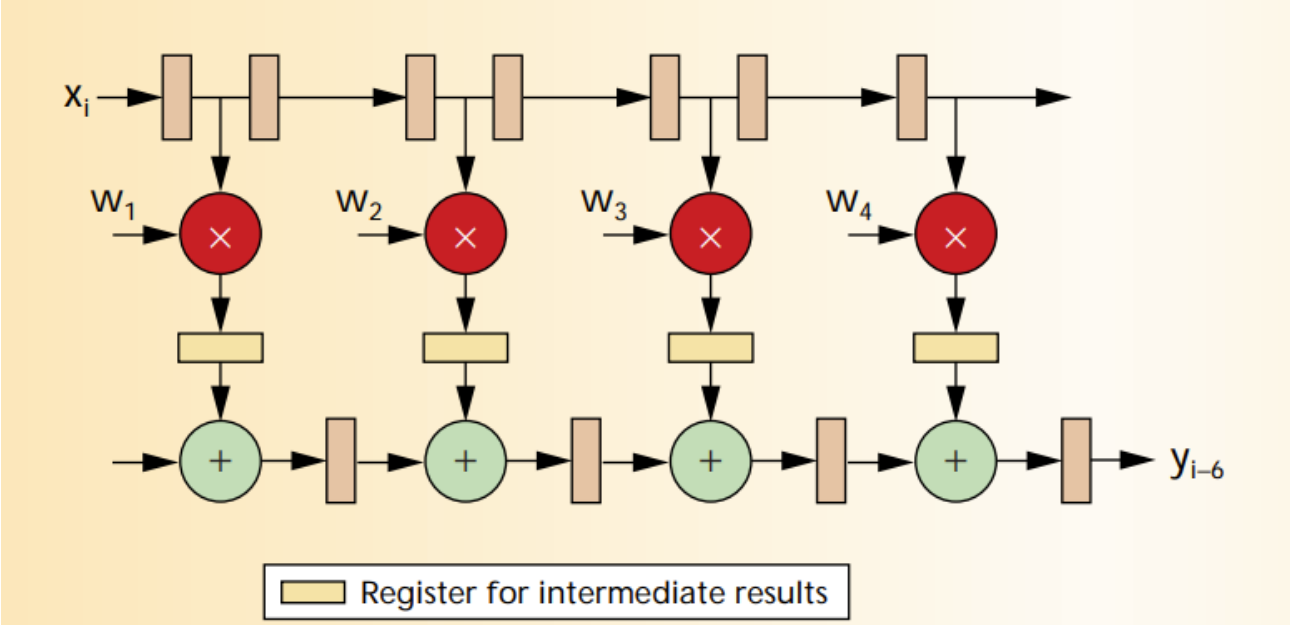
2. Shared Memory / Threads
   - Multiple threads share **same address space**; fast intra-node synchronization
   - Multicore CPU, many-core GPU SM
   - C++ threads, pthreads, OpenMP

3. Distributed memory / Processes
   - Each process has **private memory**; communication via **message passing**
   - Clusters, supercomputers
   - MPI, Charm++, Legion



Beyond the Three Levels: Pipelined Parallelism

- Pipelined / Data-Flow Parallelism (FPGAs, custom ASICs)
  - Operations are arranged in a pipeline — each stage processes a different part of the data stream concurrently.
  - Data stream in/out — minimal memory stalls
  - Low frequency → **significantly energy efficient**
  - Used in FPGAs for high-throughput, low-latency applications.



A. DeHon, *The density advantage of configurable computing*, in *Computer*, vol. 33, no. 4, pp. 41-49, April 2000, doi: 10.1109/2.839320

Implement tasks by spatially composing primitive operators—that is, by linking them together with wires.

E.g. 4-tap FIR filter : a digital filter that uses four coefficients (taps) to create an output signal that is a weighted sum of the four most recent input values

$$
\begin{aligned}
y[i] = {} & w1 \bullet x[i] + \\
& w2 \bullet x[i-1] + \\
& w3 \bullet x[i-2] + \\
& w4 \bullet x[i-3]
\end{aligned}
$$

Beyond the Three Levels: Heterogeneous Parallelism

- No single processor type optimizes *all* workloads.
  - Use **specialized hardware** for different forms of parallelism.
  - Integrate them in a single compute node or cluster.

- Heterogeneous Systems
  - Combine **CPUs** + **GPUs** [+ **FPGAs**] → each suited to a different level of parallelism.
  - E.g. Clusters of GPUs

- Challenges:
  - **Different memory spaces** → explicit data movement (CPU↔GPU, CPU↔FPGA)
  - **Load balancing** across devices with different performance characteristics
  - Varying **precision**, **bandwidth**, and **latency** requirements

Modern supercomputers are heterogeneous ecosystems — not faster clocks, but many specialized engines working together. The real challenge now isn't just parallel hardware, it's **parallel orchestration**

Parallel Scaling and Limitations

So, with an appropriately powerful computer, can we reduce the runtime to be zero ?

- Before parallelizing, we must estimate **what performance gains are realistically possible**.
  - is parallelization worth the effort ?
  - Are our best observed results reasonable ?

- **Concurrency Level** – the maximum number of computations that ***could*** run simultaneously
  - E.g. A loop with N iterations has concurrency level of N

- **Parallelism** – The **actual** number of computations running at the same time
  - Limited by hardware or overhead

Parallelism ≤ Concurrency Level

- Concurrency level is the ***potential,*** Parallelism is the ***reality***

Concurrency Level vs Parallelism example

- Consider a loop-based simulation:

```
for (int i = 0; i < N; i++) {
    work(i);   // expensive but independent work
}
```

  - Assume all iterations independent, there are no data dependencies
  - Say N = 1000, then **Concurrency Level** = 1000 : i.e. At most 1000 iterations could run at the same time
  - This is an **algorithmic** property – i.e. theoretical

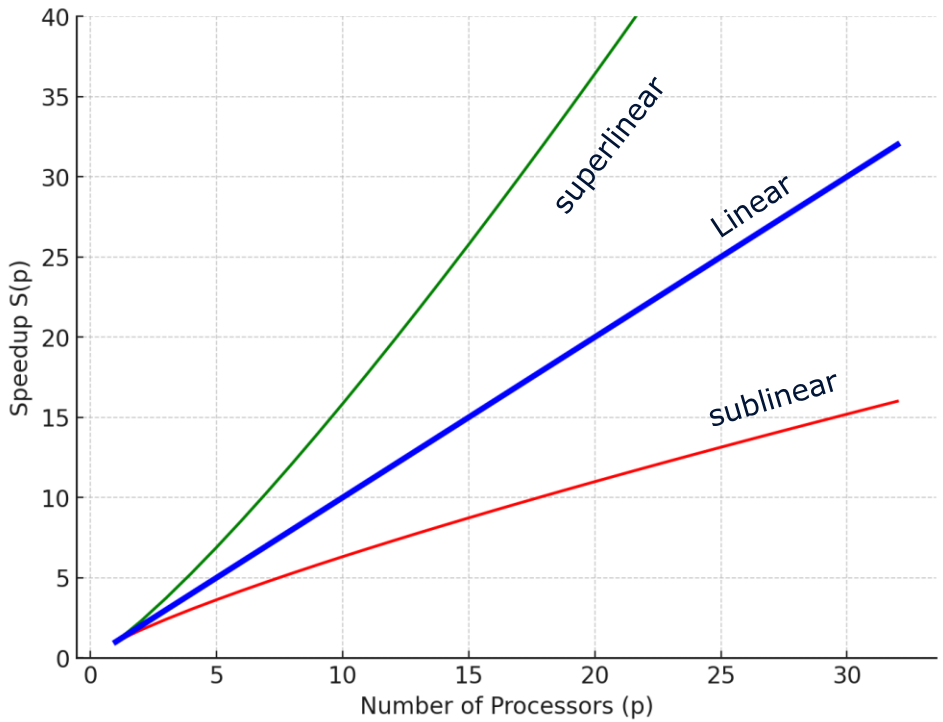
- However, in reality we have
  - Hardware constraints : e.g. The system only has 4 cores – then at most only 4 iterations can execute simultaneously – **Parallelism** = 4
  - Scheduling overhead : e.g. creating a parallel task might incur an (small but) additional overhead
  - Parallelism therefore is a observed behaviour

Speedup

- **Speedup** (S)– How much faster the program runs with p compute units:

$$S(p) = \frac{t(1)}{t(p)}$$

  - Where $t(p)$ - runtime of the code using p compute units
  - Speedup tells us how fast we can/have make/made the code
  - Speedup $S(p) \geq 0$



- $S(p) < 1$: we have invested more compute facilities, but our code has slowed down ☹
- Ideal case : $S(p) = p$ - Linear speedup ☺
- Concurrency level gives a **natural upper bound for speedup**

- $S(p) > p$: superlinear speedup, can occur due to caching effects – rare (so should be skeptical !)

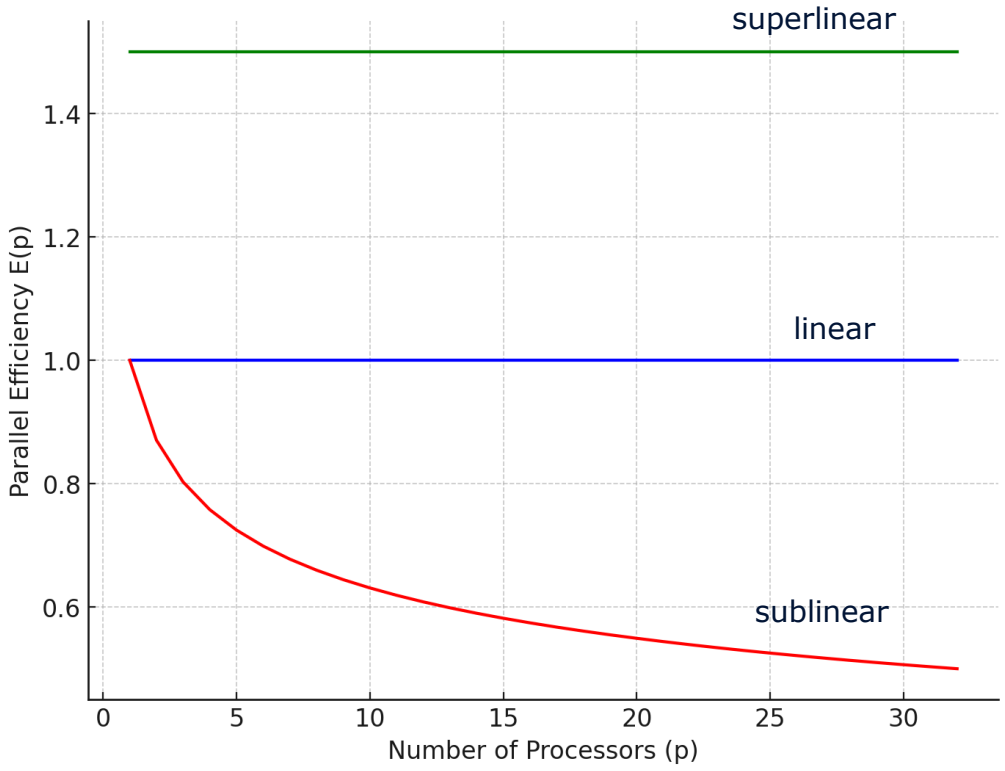
Parallel Efficiency

- **Parallel Efficiency** (*E*) - Measures *how well* the added compute units are used

$$E(p) = \frac{S(p)}{p}$$

  - Efficiency puts the scalability in relation to the number of compute units invested
  - Efficiency, $E(p) \geq 0$
  - Can also be presented as a % value

- Ideal case : $E(p) = 1$ (i.e 100%) – occurs when $E(p)$ is p



So, with an appropriately powerful computer, can we reduce the runtime to be zero ?

- In practice, we are never able to exploit all compute units all the time – there is always some work which is sequential
  - E.g. Initialization, serial I/O, atomic units of work

- Consider f as the fraction of the code that runs serially where $0 < f \leq 1$
  - Then f close to 0 means that the code keeps all compute units busy almost all the time
  - Then f close to 1 means that the majority of the code is not running in parallel
  - **Amdahl's Law (1967):**

$$t(p) = \underbrace{f.t(1)}_{\text{serial work}} + \underbrace{(1-f).\frac{t(1)}{p}}_{\text{parallel work}}$$

  - Here $f.t(1)$ cannot be parallelized and will not benefit from adding more compute units
  - The remaining fraction is shared between the compute units, which can benefit from larger *p*

Parallel Scaling and Limitations - Amdahl's Law

$$t(p) = \underbrace{f.t(1)}_{\text{serial work}} + \underbrace{(1-f).\frac{t(1)}{p}}_{\text{parallel work}}$$

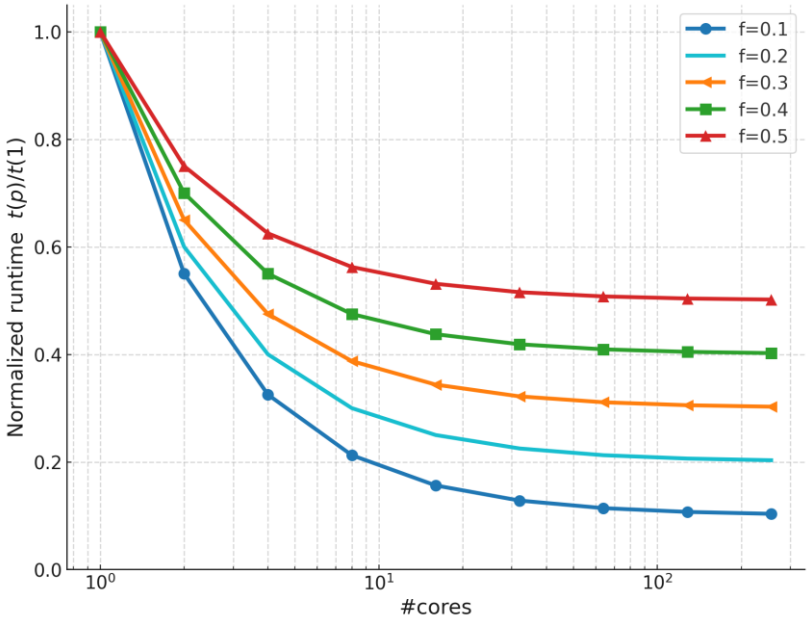
- Amdahl's Law assumes parallel and serial parts can be clearly separated
- No overhead when increasing *p*
  - in reality we have communication overhead !
- The parallel part can be split up indefinitely
  - in reality there is a point where we cannot subdivide the work

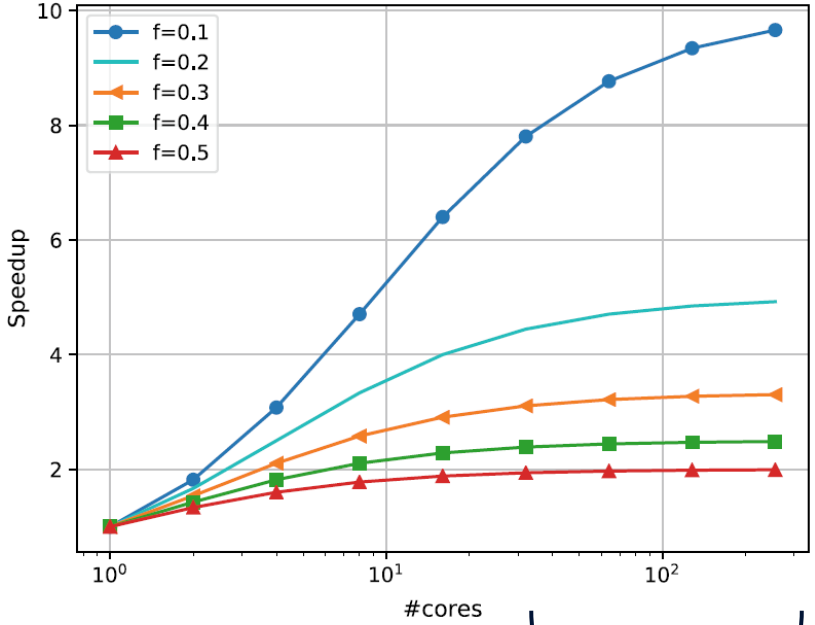
Amdal's Law – Strong Scaling

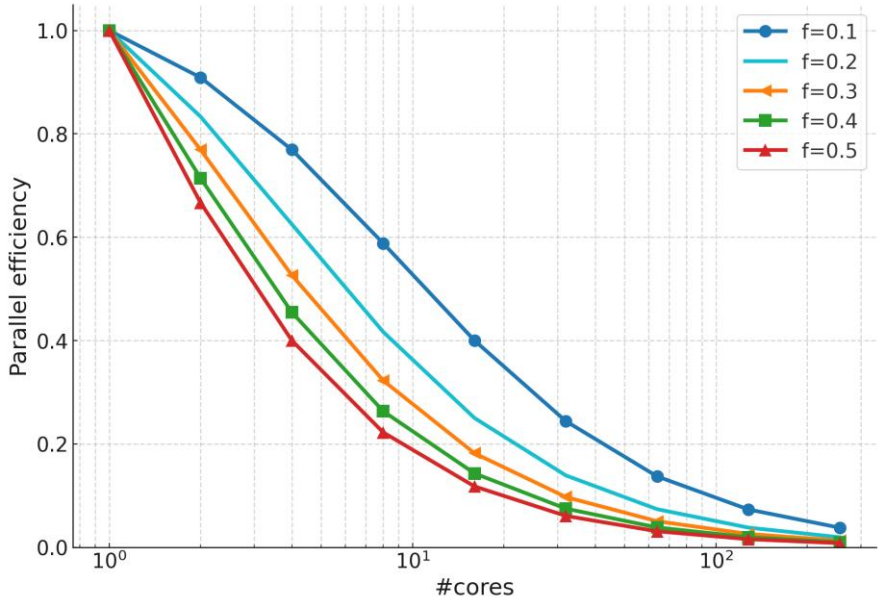
$$\frac{t(p)}{t(1)} = f + \frac{(1-f)}{p}$$

$$S(p) = \frac{t(1)}{t(p)} = \frac{1}{f+(1-f)/p}$$

$$E(p) = \frac{S(p)}{p} = \frac{1}{pf+(1-f)}$$







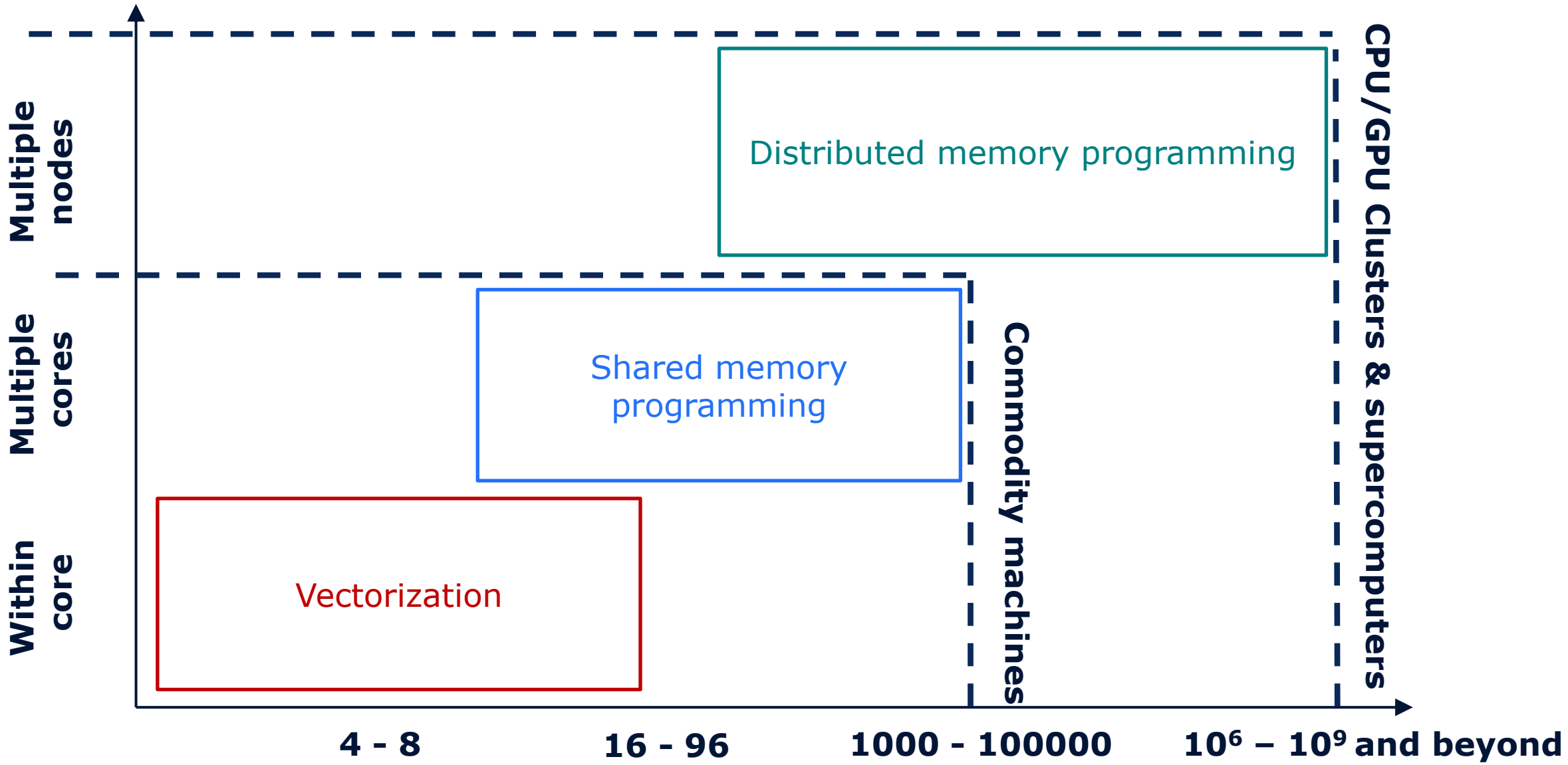
As p increases, normalized runtime converges to the serial fraction f of the code!

Diminishing returns

Diminishing returns

- Assumes that we have a fixed problem of size and solve it using bigger and bigger machines
- The f observed for a single core/node is the same for massively parallel runs - this is called ***strong scaling***

SPEEDUPS FROM THE THREE LEVELS OF PARALLELISM



GUSTAFSON'S LAW

- For a fixed problem size, **Amdahl's law presents a pessimistic view**
  - with the increase of P, the **serial portion's influence on the runtime becomes increasingly dominant**.

no amount of parallelizing will improve the performance beyond
how fast a processor can execute the serial portion of the program !

- However, if the **parallel portion is allowed to grow with the number of processors** a much more optimistic view emerges

- If the parallel portion, $(1-f)$ is increased in proportion to the number of processors p, then the time to solve the full problem on a single processor is given by:

$$t(1) = f.t(p) + (1-f).t(p).p$$

$$S(p) = \frac{t(1)}{t(p)} = f + p(1-f)$$

- Assume a problem already running in parallel on p processors taking time $t(p)$
- $f.t(p)$ is time taken by serial part
- $(1\text{-}f).t(p)$ is time taken by parallel part
- Then running the same workload on 1 processor will need to do p processors work sequentially, i.e. $t(p)$

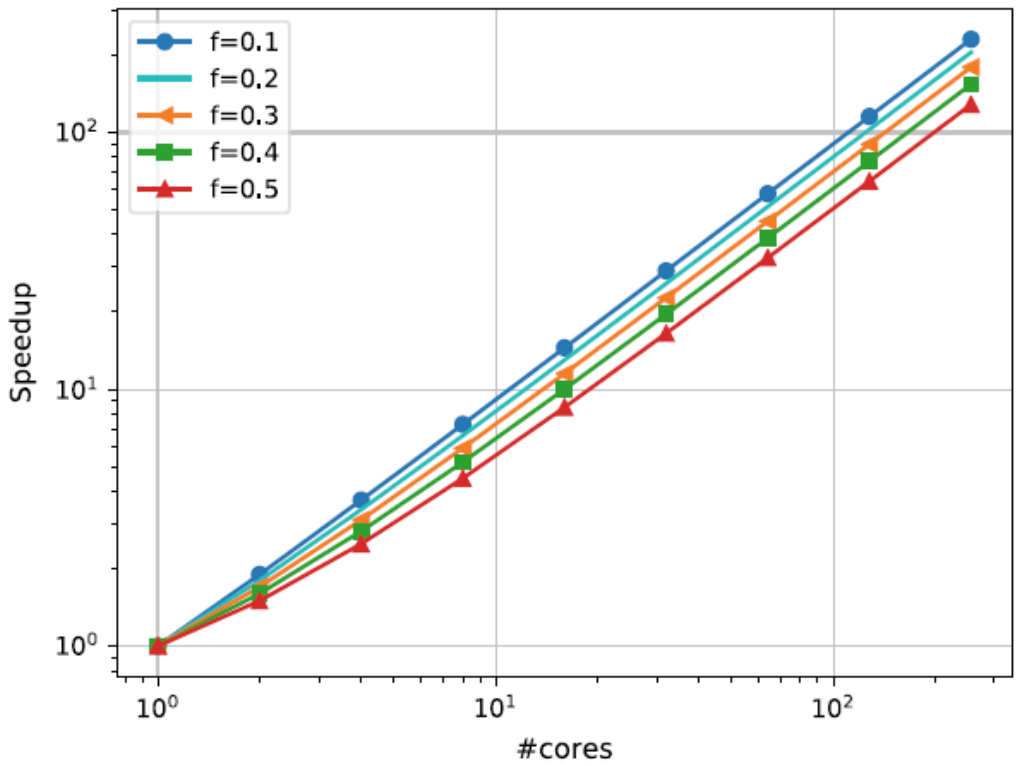
Gustafson's Law

$$t(1) = \underbrace{f.t(p)}_{\text{serial work}} + \underbrace{(1-f).t(p).p}_{\text{parallel work}}$$

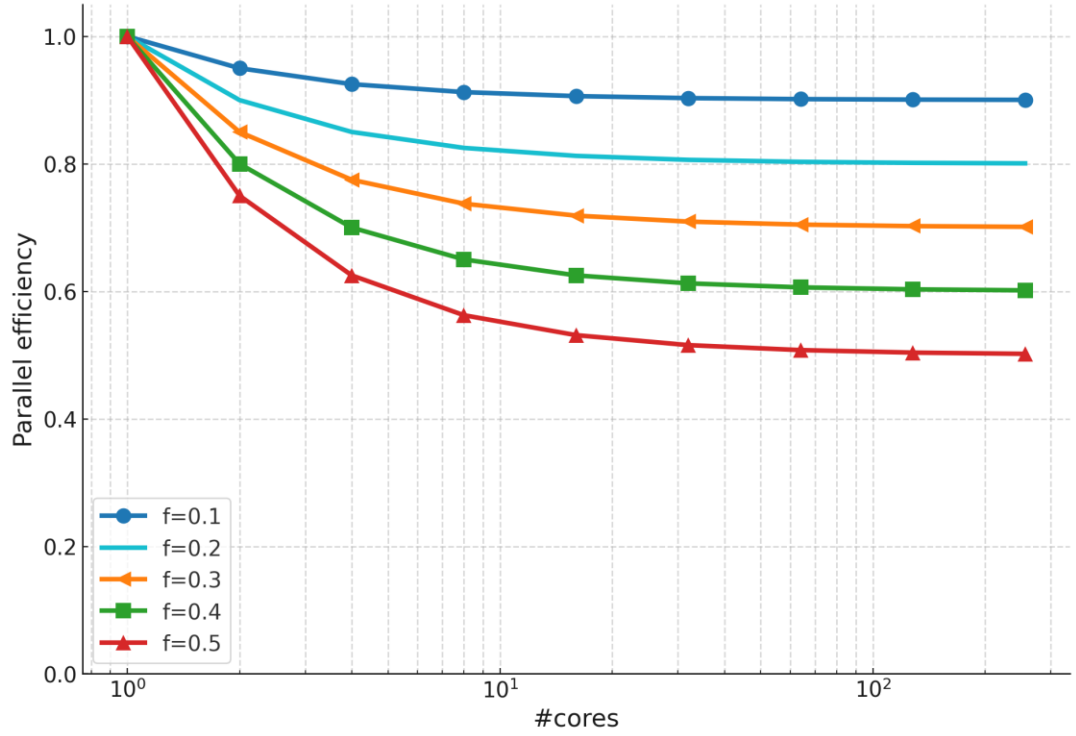
- Gustafson's law also assumes parallel and serial parts can be clearly separated
- No overhead when increasing p
  - in reality we have communication overhead !
- The parallel code's serial part f remains the same for a serial re-run
  - Implies that any parallel code we start from has a sufficiently high concurrency level

Gustafson's Law – Weak scaling

$$S(p) = \frac{t(1)}{t(p)} = f + p(1-f)$$



$$E(p) = \frac{S(p)}{p} = (1-f) + \frac{f}{p}$$



As p increases, parallel efficiency converges to the parallel fraction $(1-f)$ of the code!

- We increase the problem size as we increase the number of processors - this is called ***weak scaling***
- The serial portion reduces relative to the parallel portion

Scaling examples from Real-world codes – OP2-Hydra

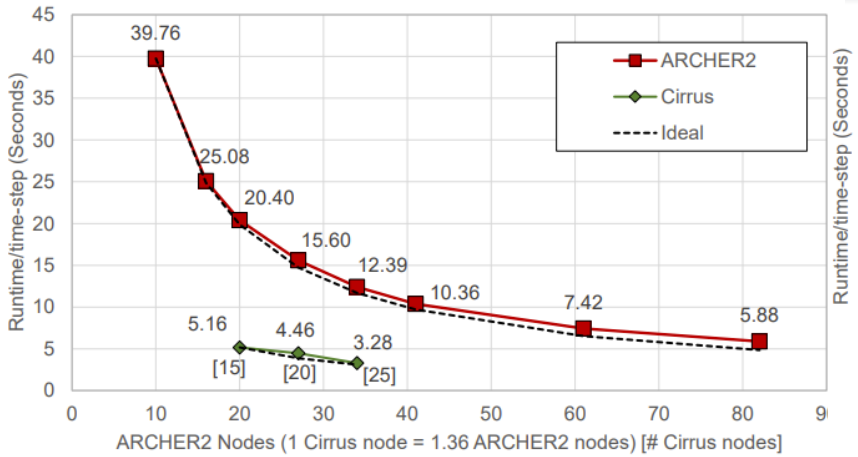


Fig. 7: Rig250 $1 - 10_{430M}$ Mesh Runtime.

3.7- 4x speedup

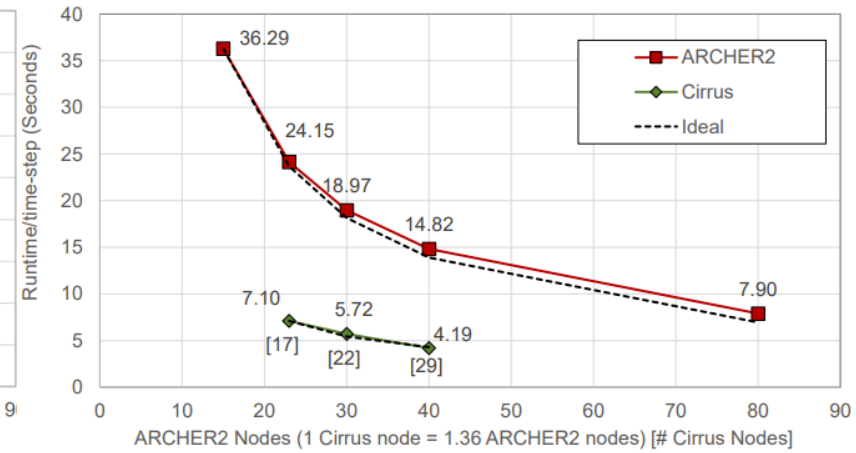


Fig. 8: Rig250 $1 - 2_{653M}$ Mesh Runtime

3.3 - 3.4x speedup

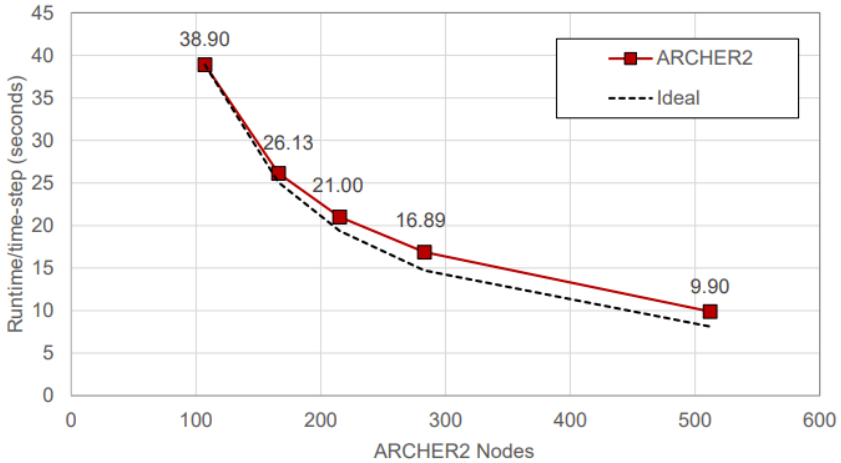


Fig. 9: Rig250 $1 - 10_{4.58B}$ Mesh Runtime

- DLR – Rig250 Compressor
  - 430M, 653M and 4.58B meshes
- 4.58B mesh - ARCHER2 @ 512 nodes: 82% parallel efficiency (vs 107 node run)

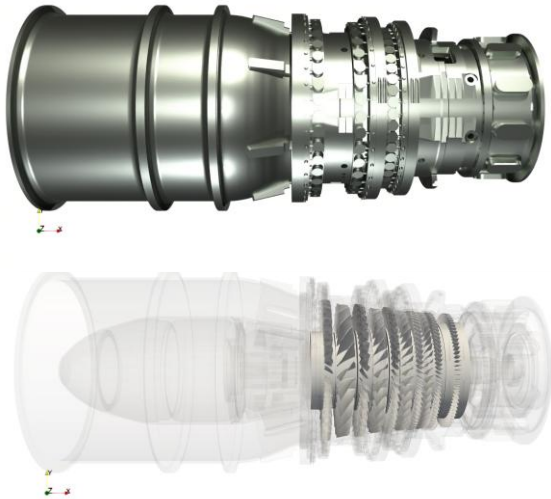


TABLE I: Systems specifications.

System	ARCHER2 HPE Cray EX	Cirrus SGI/HPE 8600 GPU Cluster
Processor	AMD EPYC 7742 @ 2.25 GHz	Intel Xeon Gold 6248 (Cascade Lake) @ 2.5 GHz + NVIDIA Tesla V100-SXM2-16GB GPU
(procs×cores) /node	2×64	2×20 + 4×GPUs
Memory/node	256 GB	384 GB + 40GB/GPU
Interconnect	HPE Cray Slingshot 2×100 Gb/s bi-directional/node	Infiniband FDR, 54.5 Gb/s
OS	HPE Cray LE (based on SLES 15)	Linux CentOS 7
Compilers	GNU 10.2.0	nvfortran (nvhpc 21.2)
Compiler Flags	`-O2 -eF -fPIC`	CUDA 11.6 and sm_70 -O2 -Kieee
Power/node	660W	≈ 900W

G.R. Mudalige, I.Z. Reguly, A. Prabhakar, D. Amirante, L. Lapworth, S.A. Jarvis, *Towards Virtual Certification of Gas Turbine Engines With Performance-Portable Simulations.* In 2022 **IEEE International Conference on Cluster Computing (CLUSTER)**, 2022

READING

- **Herb Sutter**: *The Free Lunch Is Over—A Fundamental Turn Toward Concurrency in Software*. Dr. Dobb's Journal. 30(3) (2005) http://www.gotw.ca/publications/concurrency-ddj.htm
- **Gordon E. Moore**: *Cramming more components onto integrated circuits*. Electronics Magazine. 38(8), pp. 33–35 (1965) https://doi.org/10.1109/N-SSC.2006.4785860
- **Robert H. Dennard et al.**: *Design of ion-implanted MOSFET's with very small physical dimensions*. IEEE Journal of Solid State Circuits. 9(5), pp. 256–268 (1974) https://doi.org/10.1109/JSSC.1974.1050511
- **Michael J. Flynn**: *Some Computer Organizations and Their Effectiveness*. IEEE Transactions on Computers. C-21(9), pp. 948–960 (1972) https://doi.org/10.1109/TC.1972.5009071
- **A. DeHon**, *The density advantage of configurable computing*, in *Computer*, vol. 33, no. 4, pp. 41-49, April 2000, doi: 10.1109/2.839320

- **Gene M. Amdahl**: *Validity of the single-processor approach to achieving large scale computing capabilities.* Proceedings of the American Federation of Information Processing Societies. AFIPS Press, pp. 483–485 (1967) https://doi.org/10.1145/1465482.1465560
- **John L. Gustafson**: *Reevaluating Amdahl's law*. Communications of the ACM (31), pp. 532–533 (1988) https://doi.org/10.1145/42411.42415